



Learning-based dynamic cache management in a cloud

Jinhwan Choi^a, Yu Gu^b, Jinoh Kim^{a,*}

^a Texas A&M University, Commerce, TX 75428, USA

^b Visa Inc., Austin, TX, USA



ARTICLE INFO

Article history:

Received 7 July 2018

Received in revised form 17 March 2020

Accepted 20 June 2020

Available online 3 July 2020

Keywords:

Dynamic caching

Cloud cache management

Access pattern estimation

Cache size prediction

Machine learning

ABSTRACT

Caches are an important component of modern computing systems given their significant impact on performance. In particular, caches play a key role in the cloud due to the nature of large-scale, data-intensive processing. One of the key challenges for the cloud providers is how to share the caching capacity among tenants, under the circumstance that each often requires a different degree of quality of service (QoS) with respect to data access performance. The invariant is that the individual tenants' QoS requirements should be satisfied while the cache usage is optimized in a system-wide manner. In this paper, we introduce a learning-based approach for dynamic cache management in a cloud, which is based on the estimation of data access pattern of a tenant and the prediction of cache performance for the access pattern in question. We consider a variety of probability distributions to estimate the data access pattern, and examine a set of learning-based regression techniques to predict the cache hit rate for the access pattern. The predicted cache hit rate is then used to make a decision whether reallocating cache space is needed to meet the QoS requirement for the tenant. Our experimental results with an extensive set of synthetic traces and the YCSB benchmark show that the proposed method consistently optimizes the cache space while satisfying the QoS requirement.

© 2020 Elsevier Inc. All rights reserved.

1. Introduction

Caches are an important component of modern computing systems given their significant impact on the data access performance, which is more critical in a large-scale data processing environment. For example, leading Web service providers such as Google, Facebook and Amazon largely rely on in-memory processing using Memcached [15,21] and Redis [29]. Even small-scale providers employ the caching services through Amazon ElastiCache [3], Memcachier [25], and so forth. This is because that the efficient use of in-memory caching has a significant impact on the performance as the reading from the storage is much more expensive than from the cache. In addition to the performance benefits, using a cache reduces the load in the back-end data servers giving greater scalability. Since the cloud often requires processing massive data (e.g., for hosting Web or e-Commerce services), in-memory caching is an essential part to improve the data access performance.

As the number of users and network traffic increases, service providers may need to expand the infrastructure. However, adding hardware resources infinitely for better performance is costly and impossible. Thus, it is important to use the limited

resources more in an efficient way. In that sense, cache management plays a key role to better utilize the limited cache spaces, in order to maximize the performance and efficiency of data access. Cache management consists of a set of functions including cache space (re)allocation and eviction. In this work, we focus on the problem of cache space optimization under the assumption of the use of in-memory caching in a cloud.

A crucial concern for cache management in a cloud is that the caches are shared by multiple tenants that may have different quality of service (QoS) requirements with respect to data access performance. Thus, it is necessary to meet the per-tenant requirement in addition to the optimal use of the cache in a system-wide manner. A simple technique to deal with this problem is *static caching* that allocates the exclusively dedicated cache blocks to tenants based on the individual QoS specifications. However, it may not be straightforward to determine how many cache blocks would be needed for a tenant without having the knowledge of data access pattern. Moreover, the access pattern can be changed over time, and the initially assigned cache blocks can be too small violating the performance requirement or too large causing the waste of the limited system resource. *Dynamic caching* (re)allocates cache space based on the demand of tenants dynamically, and it would be beneficial not only to meet the per-tenant requirement but also to improve the overall system performance. Hence, it is essential to properly capture the demand of cache space when using dynamic caching. Another alternative is to use

* Corresponding author.

E-mail addresses: jchoi8@leomail.tamuc.edu (J. Choi), yugu1@visa.com (Y. Gu), jino.kim@tamuc.edu (J. Kim).

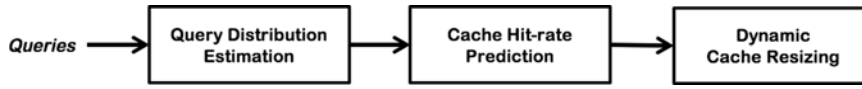


Fig. 1. Process model for dynamic cache management.

a global cache shared among tenants but isolation guarantee, which is one of the desirable properties in a cloud, cannot be provided [26,33]. In this work, we take dynamic caching into account to deal with the problem of cache management with the benefit of low overhead in the cache replacement time since the interval of cache re-allocation is relatively infrequent and does not need to be performed at every eviction.

For dynamic caching in a cloud, there has been a body of work in the past few years, mainly by estimating the cache performance of individual tenants (or applications) [10,11,13,26]. However, previous studies are largely limited with the lack of well-defined models for the estimation of data access patterns and the prediction of cache performance for a given access pattern. With the recent disruptive advance of machine learning technologies, we take a learning-based approach to cache management in a cloud. To estimate data access patterns varied over time, we consider a set of probabilistic distributions (uniform, Gaussian, exponential, and Zipf). The information of the estimated access pattern is then used to predict the cache size to meet the specified data access performance, and we assume the QoS is defined with the required cache hit rate. For this purpose, we examine various regression methods, including Support Vector Regression (SVR), Gaussian Process Regression (GPR), and Fully Connected Neural Network (FCN). Fig. 1 shows our process model to implement dynamic caching in a cloud with the aforementioned functions of access pattern estimation and cache performance prediction. For a query-based transaction system, we assume that the access pattern is inferred based on the query key distribution. From the estimated distribution, cache hit rate is predicted to determine the right size of cache space to meet the given QoS requirement. Finally, cache resizing takes place if needed.

The key contributions of this paper are summarized as follows:

- We formulate a problem of dynamic caching in a cloud, with two objectives of QoS guarantee and optimization of the cache space in the system;
- We present a method for the query distribution estimation with a set of probabilistic distributions including normal, Gaussian, exponential, and Zipf. We demonstrate that our estimation method using the Kolmogorov–Smirnov Test (KS-Test) [12] can yield accurate estimations even with a small number of query samples (≤ 1000 samples), which is beneficial for responding to the temporal pattern changes in a timely manner;
- We examine a set of learning-based regression techniques including SVR, GPR, and FCN, to predict the cache hit rate based on the estimated query distribution. Our evaluation supports the effectiveness of the FCN model across the diverse distributions with respect to the prediction performance;
- We evaluate our proposed process for dynamic caching with an extensive set of synthetic traces and Yahoo! Cloud Serving Benchmark (YCSB) [14], on a cluster computing system with Open Source Apache Ignite [4] as the in-memory caching infrastructure. The evaluation results confirm that the proposed method consistently optimizes the cache space, maintaining the required cache hit rate as the QoS requirement.

This paper is organized as follows. In the next section, we specify the description of the problem tackled in this study and provide a summary of the closely related studies in the area of cache management in the cloud. Section 3 presents our proposed process model with the details of the functional elements. We report the evaluation results in Section 4 conducted with the synthetic traces and YCSB benchmark tools. Finally, we conclude our presentation in Section 5 with a summary of the work and future directions.

2. Background

2.1. Problem statement

In this work, we assume a cloud system equipped with a large-scale in-memory cache. A tenant loads their data to the cloud and accesses them. We assume a read-many, write-rare environment through key–value stores like a database transaction system. Hence, the cache performance is a predominant factor for the overall system performance. The tenant is associated with an agreement (SLA) that specifies a set of QoS requirements including data access performance. In this study, we focus on the cache hit rate to measure the data access performance.

To formulate, we use the notation defined in Table 1. A set of tenants $T = \{T_1, T_2, \dots, T_n\}$ exist in the cloud with the total cache space of C . Initially, a cache space C_{T_k} is allocated to tenant T_k , which can be adjusted over time to meet the specified cache hit rate requirement (H_{T_k}). The current cache hit rate (h_{T_k}) is periodically measured, and our objective is to keep $h_{T_k} \geq H_{T_k}$ for the tenant.

It would be easy to guarantee H_{T_k} by simply allocating a plenty of cache space to the tenant which would be much greater than the actual need. However, there will be a significant waste of the expensive cache resource in that case. In order to perform the system-wide optimization as well as the minimum QoS guarantee for individual tenants, this work addresses the following optimization problem:

$$\begin{aligned}
 &\text{Minimize} && \sum_{T_k \in T} C_{T_k} \\
 &\text{such that} && \sum_{T_k \in T} C_{T_k} \leq C \\
 &\text{and} && h_{T_k} \geq H_{T_k}, \forall T_k
 \end{aligned}$$

If the system has insufficient free cache space to allocate to a certain tenant, we assume that it is reported to the administrator who conducts the defined policy to deal with such an exceptional situation.

2.2. Data access patterns

In this work, we basically employ Zipfian distributions to model data access patterns. A body of studies reported Zipf-like patterns for data access through the measurement studies [1,6,7,9,16,19,31,32]. For example, Zipf ($\alpha = 0.9$) was assumed to model the load distribution for caching [28], while $\alpha = 0.99$ was used for the key–value request workload in a cloud [30]. The work in [23] assumed $\alpha = 1.0$ for the Zipf-like file accesses in their evaluation. In addition, the tool of Yahoo! Cloud Serving Benchmark (YCSB) [14] is widely employed for modeling cloud

Table 1

Notation.

Notation	Description
T_k	Tenant k in the tenant set T ($T_k \in T$)
C	Total cache space in the system
C_{T_k}	Cache size allocated to T_k
h_{T_k}	Measured cache hit rate for T_k
H_{T_k}	Minimum cache hit rate requirement for T_k
$H(x, y)$	Predicted cache hit rate under cache size x and access distribution y
δ	Safety margin

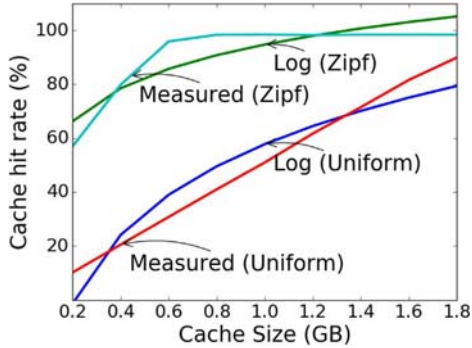


Fig. 2. Cache performance estimation using a log function-based least-square fit: This concave function may not be adequate for cache performance prediction with large errors over 10% gaps to the measured result.

workloads and is based on Zipfian distributions. In this work, we assume Zipf-based workloads to model data access patterns with a range of coefficient values ($0.5 \leq \alpha \leq 3.0$), to consider more extreme patterns as well.

Besides, several studies assumed non-Zipf patterns to model data access patterns. For instance, exponential distributions were assumed for the reference rank model of Internet media objects [20]. Another study employed an exponential model can be found in [8] to model mobility patterns for mobility-aware caching for cellular communication. For the possibility of non-Zipf accesses, we also consider uniform and, Gaussian distributions, as well as exponential patterns in this study.

2.3. Related work

There exist several interesting studies for in-memory caching in the cloud environment. This section summarizes the most relevant studies to this research with the differences from our work.

Blaze [10] is the first generation work for the data caching in a cloud with the consideration of multiple tenants. The main objective of this work is not only to guarantee the minimum QoS requirement for each tenant but also to maximize the overall cache hit rate. To meet the tenant's QoS goal, the authors proposed to estimate the cache hit rate using a log function ($y = a + b \log x$) based on a least-square fit, where the output (y) is the predicted cache hit rate from the given cache size (x). In this scheme, determining the parameters (a and b) is a non-trivial challenge. More critically, we observed considerable errors from our preliminary experiment, as shown in Fig. 2 that compares the measured hit rates and the log-based fitted result for two different access patterns. The figure shows over 10% errors for certain predictions. As will be demonstrated in Section 3, it would be hard to fit the hit rates using a simple concave function, and we employ learning-based techniques to provide greater accuracy and flexibility.

There have been some studies utilizing the concept of stack distance [2] to estimate the data access pattern for the cloud cache management. In the SC2 work [11], a cache space utility model is introduced to maximize the overall cache hit rate in a multi-tenant environment. The cache space utility model is a cumulative distribution function of the stack distance hit histogram. The cumulative curve is then used to figure out the minimum cache space that satisfies the required minimum cache hit rate. A critical problem referencing to stack distances to estimate data access patterns is the heavy complexity to keep shadow eviction queues for incoming requests, in order to track the location of the request to calculate the stack distance.

Dynacache [13] has been proposed to improve the cache hit rates in a multi-application environment. This technique also relies on stack distances to infer the cache hit rate curve, and interestingly, it uses an approximation technique using buckets to reduce the overhead for calculating stack distances. However, the estimation is still expensive when assuming hundreds of applications running in a system. Dynacache defines a metric to identify applications that can be more benefited from the cache management and stack distances are estimated only for a small set of the identified applications. While beneficial to reference to the stack distance information, it is heavy to calculate and even the approximation is still expensive. We do not rely on stack distances, but employ a set of probability models to estimate tenants' access patterns, with a distribution comparison function. This past work also assumes a simple concave function to predict cache hit rates from the given cache size.

A recent work FairRide [26] tackles a cheating problem that may lead to monopolizing the cache resource by a greedy user. The cheating problem can happen in an environment that the users often access the same data (e.g., utility programs). For example, a greedy user can get a free ride by accessing the shared files already in the cache space loaded by other users. The authors proposed a simple idea to mitigate this problem by imposing a penalty (e.g., delay) to such users. The work in [33] further studies on the problem of fair cache allocation for in-memory analytics to prevent from free-riding manipulations that may result in poor cache utilization. In this work, we do not deal with a cheating problem, but it may be an interesting topic for future exploration.

3. Proposed design

In this section, we present our design to tackle the problem stated in the previous section. Fig. 3 shows the proposed architecture for our learning-based dynamic cache management in a cloud. The overall scenario to meet the QoS requirement (i.e., cache hit rate) for each tenant is as follows. *Pattern Estimator* estimates the distribution of query keys sampled by *Profiler* for each tenant periodically (e.g., at every k number of samples). Once the estimation result is available, *Predictor* makes a prediction using a regression function to calculate the tenant's cache hit rate with the currently allocated cache size C_{T_k} and the estimated probability distribution d . In Table 1, we define the predicted cache hit rate for tenant k as $H(C_{T_k}, d)$. Then the predicted hit rate is compared to the minimal requirement (H_{T_k}). Based on

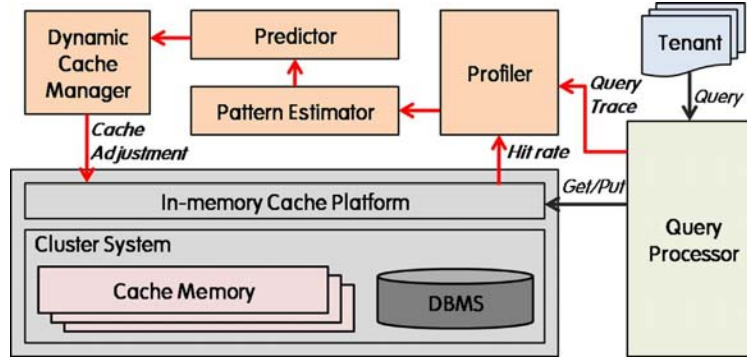


Fig. 3. Architecture for dynamic cache management: *Pattern Estimator* estimates the distribution of query keys sampled by *Profiler* for each tenant periodically. Once an estimated distribution information is available, *Predictor* makes a prediction using a regression model to calculate the tenant's cache hit rate with the currently allocated cache size and the estimated probability distribution. *Cache Manager* determines whether the tenant's cache needs to be resized or not, based on the prediction result.

Table 2
Distribution parameter values.

Distribution	Parameter value range	Step parameter
Uniform	N/A	N/A
Gaussian	$0.5 \leq \sigma \leq 2.0$	$s = 0.1$
Exponential	$0.5 \leq \lambda \leq 2.0$	$s = 0.1$
Zipf	$0.5 \leq \rho \leq 3.0$	$s = 0.1$

the comparison result, *Cache Manager* determines whether the tenant's cache needs to be resized or not. We next discuss the functional components in the proposed model in detail.

3.1. Query distribution estimation

Once a tenant begins to access her data, the system starts estimating the distribution of the keys. A hypothesis here is that the key distribution approximates one of the probabilistic distributions we consider in this study, including uniform, Zipf, exponential, and Gaussian, as discussed in Section 2.2.

The estimation of distribution takes place through the KS test, which has been broadly employed for comparing empirical distributions [12]. In the KS test, the resulted p -value indicates the similarity of the two distributions in question. In our context, two samples in comparison are: (1) the collection of query samples for a tenant, and (2) the synthetic samples derived from the tenant's key space using a distribution model. A higher p -value indicates the tenant access pattern is closer to the synthetic distribution in comparison.

Estimating the degree of uniformity is straightforward and can be done through a single KS test, since any sample set from the uniform distribution have the identical property. In contrast, measuring the similarity against the non-uniform distributions would be complicated and the property of the samples can be varied by the distribution-specific parameter. For example, a Zipf distribution becomes unique with parameter ρ , based on $y \sim x^{-\rho}$. Similarly, the exponential distribution has parameter λ from $y \sim e^{-\lambda x}$, and Gaussian is defined with a variance σ in the standard distribution where $\mu=0$. We consider a broad range of values for the distribution-specific parameters for greater accuracy in the estimation process, as summarized in Table 2. From the table, the parameter values are ranged from the min to max, which determines the skewness of the distribution. For example, higher ρ and λ values indicate greater skewness for Zipf and exponential, respectively. In contrast, a lower σ implies a greater skewness for Gaussian.

To measure the similarity against the non-uniform distributions, we define a step parameter $s = 0.1$ as the granularity in comparison. For each non-uniform distribution, a KS test is arranged for a specific parameter value, from the min to max value increased by s . For example, the estimation process runs 16 independent KS tests to measure the similarity against the Gaussian distribution with a σ value from 0.5 to 2.0, incrementing it by $s=0.1$. The estimation process continues with the exponential and Zipf distributions with their distribution parameter and s . Finally, a distribution with the greatest similarity is chosen as the estimated distribution to model the tenant's access pattern. If the KS test fails to identify a candidate distribution for the given access samples, the uniform distribution is assumed as the access pattern that requires the largest cache space compared to the other distributions as the fallback scenario.

Intuitively, using a larger number of query samples would be helpful for estimating the access pattern more accurately. Table 3 shows the impact of the number of samples in the estimation process. In the table, ϵ stands for the difference between the actual distribution parameter value (p) and the estimated one ($\hat{p}$) (i.e., $\epsilon = |p - \hat{p}|$). As seen from the table, it is possible to estimate the distributions correctly only with 200 samples. However, 15% of the estimations show $\epsilon > 0.2$, which may lead to a non-negligible error in the next stage for predicting cache hit rates. It becomes quite stable with 1000 samples or more, and 100% of the estimates are bounded to $\epsilon \leq 1.0$. Since we assume a high access rate (e.g., eCommerce services), collecting 1000 samples would not be a big deal and could be made within a short time interval.

3.2. Cache hit rate prediction

Once a query distribution is estimated, the next step in the process model is to predict the cache performance for the tenant in question. Depending on the prediction result, the tenant's cache space can be adjusted to keep up with the data access pattern changes over time. A high degree of accuracy for the prediction is thus essential and the tenant's QoS requirement may not be met otherwise. In this section, we examine a set of regression techniques including SVR, GPR, and FCN, with the metric of MSE (Mean Squared Error) to evaluate the regression performance. For thorough analysis, we employ an extensive set of data sizes and distribution parameter values, summarized in Tables 4 and 5. Note that the data sets for training and testing are *disjoint* without any overlaps, as can be seen from the tables. We also examine the models under the assumption of different

Table 3
Impact of the number of samples to KS test.

# Samples	100	200	300	500	1000	2000	5000	7000	10000
Correct dist.	97%	100%	100%	100%	100%	100%	100%	100%	100%
Exact param.	58%	53%	68%	66%	76%	81%	93%	93%	97%
$\epsilon \leq 0.1$	75%	78%	93%	90%	100%	100%	100%	100%	100%
$\epsilon \leq 0.2$	90%	85%	95%	98%	100%	100%	100%	100%	100%

Table 4
Learning data for cache hit rate prediction.

Distribution	Training	
	Data size	Parameter values
Uniform	{1GB, 2GB, 4GB, 8GB}	N/A
Gaussian(σ)	[1GB..9GB] incremented by 1GB	{0.5, 1.0, 1.5, 2.0}
Exponential(λ)	[1GB..9GB] incremented by 1GB	{0.5, 1.0, 1.5, 2.0}
Zipf(ρ)	[1GB..9GB] incremented by 1GB	{0.5, 1.0, 1.5, 2.0, 2.5, 3.0}

Table 5
Testing data for cache hit rate prediction. Note that the testing data sets are *disjoint* from the training data sets in Table 4 with no overlaps.

Distribution	Testing	
	Data size	Parameter values
Uniform	{3GB, 6GB}	N/A
Gaussian(σ)	[1 GB..9 GB] by 1 GB	{0.7, 1.2, 1.9}
Exponential(λ)	[1 GB..9 GB] by 1 GB	{0.7, 1.2, 1.9}
Zipf(ρ)	[1 GB..9 GB] by 1 GB	{0.7, 1.2, 1.9, 2.3, 2.6}

cache sizes from 0.1 GB to 4 GB. For simplicity, we assume four bytes for the key field and 100 KB for the value stored in the key-value repository; for example, there exist 10,240 unique keys if the data size is 1 GB (i.e., $\frac{1 \text{ GB}}{100 \text{ KB}}$).

To measure actual cache hit rates, we set up a cloud testbed using five servers in a cluster computing system installed with Apache Ignite as the in-memory caching infrastructure. Three nodes are assigned for the cache service, and one node each for a client and backend database server. The detailed information regarding our experimental settings can be found in Section 4.1.

3.2.1. SVM regression (SVR)

SVM has been widely employed for classification and regression [5,27,34]. To predict the cache hit rate, the first regression model we examined is SVM regression (SVR). We assumed the Radial Basis Function (RBF) kernel and the standard scaler for normalization.¹ A tricky part using SVR is to tune a set of parameters to optimize. We examined SVR with an extensive set of the values for the penalty parameter (C) and the kernel coefficient parameter (γ): $10^{-3} \leq C \leq 10^7$ and $10^{-5} \leq \gamma \leq 10^3$.

Fig. 4 shows the regression performance (with respect to MSE) with the combinations of the parameter values in SVR. Lighter colors indicate smaller errors. The figure shows that it needs to optimize the parameters for each distribution independently and there exists no single pair of parameters working well for all the distributions. We observed that SVR works better for heavy-tail distributions (i.e., Zipf and exponential distributions) showing $\text{MSE} < 5.0$ with the parameter tuning, but poorly works for the uniform distribution with large errors ($\text{MSE} > 22.0$) even at best.

3.2.2. Gaussian process regression (GPR)

GPR has also been utilized for regression [18] and we take GPR into account to predict cache hit rates for individual distributions. We set it up with two kernels of a constant kernel configured with Constant Value (CV) and a RBF kernel tuned with Length Scale (LS) [17]. The value ranges for CV and LS to search the optimal are: $0.0001 \leq \text{CV} \leq 1000$ and $0.0001 \leq \text{LS} \leq 1000$.

¹ We examined both standard and MinMax scalers for SVR and GPR, and observed none of them outperforms the other completely.

Fig. 5 shows the regression performance when using GPR across the parameter values. From the figure, LS shows a greater impact to the regression performance, and $\text{LS} \geq 1.0$ works much better than the other. The trend is similar with SVR, showing lower prediction errors for the heavily skewed distributions. Overall, we observed no better performance from GPR compared to SVR.

3.2.3. Regression with fully connected network (FCN)

The recent advances in deep learning has led to an introduction of many useful tools to implement learning models, promoting a wide adoption of the relevant techniques for many applications that require analyzing the data with a non-linear property. Another regression model we investigate in this study is based on a fully connected neural network (FCN in short). We examined the FCN architecture with the following considerations to evaluate their impacts on the accuracy of prediction:

- **Network architecture:** To keep it simple, we designed a basic form of the FCN architecture with a single hidden layer. The number of neurons in the hidden layer examined in the experiment is {16, 32, 64, 128, 256}, while we set the number of neurons in the input layer to 20.
- **Loss functions and regularization:** Kernel regularizers play an important role to calculate penalties on layer parameters, which are then combined in the loss function to optimize the network. We evaluated L1 and L2 regularizers. We did not apply dropout and the expansion of training data for regularization. For the loss function, we compared Mean Absolute Error (MAE) and MSE (Mean Squared Error).
- **Activation function:** The activation function is the non-linear transformation that determines which neurons are activated or not. We tested Sigmoid and ReLU, widely used in practice.
- **Number of epochs:** In an epoch, the training process performs forward and backward propagation over the entire dataset. Typically, a higher number of epochs is required for a complex dataset with a longer training time, and vice versa. We examined the impact of the number of epochs by setting the epoch time to one of {500, 1000, 2000, 4000}.

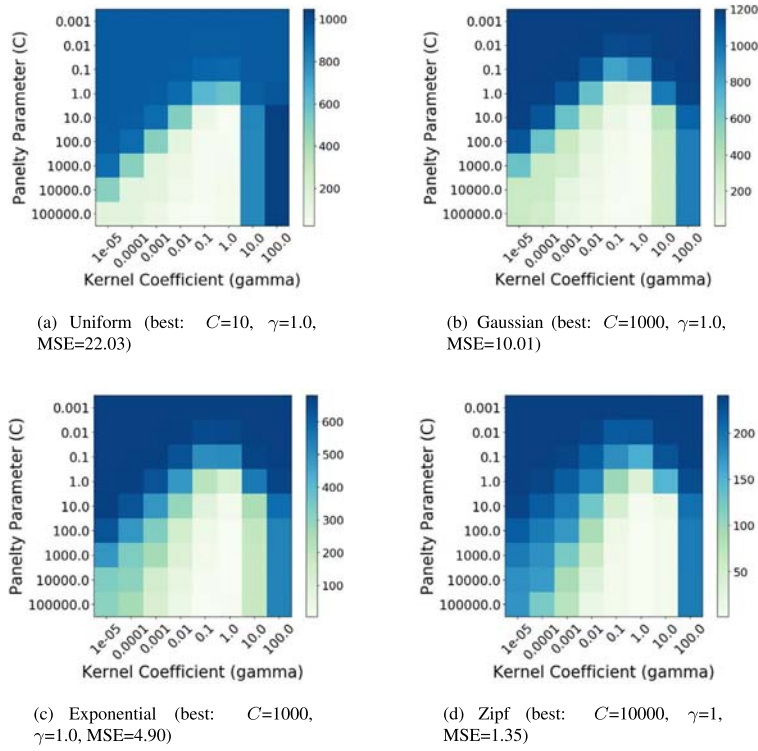


Fig. 4. SVR parameter tuning for cache hit rate regression: C =penalty parameter and γ =kernel coefficient parameter.

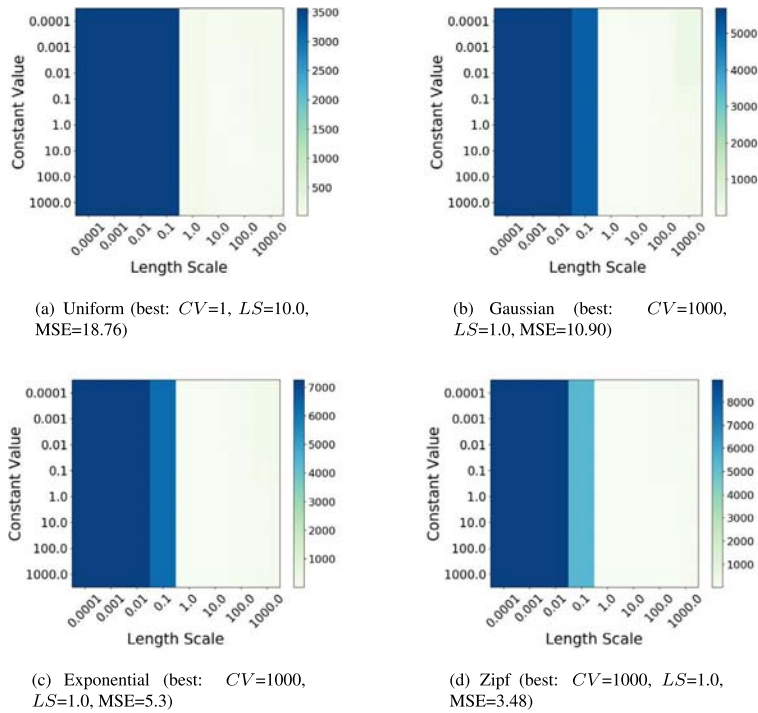


Fig. 5. GPR parameter tuning for cache hit rate regression: CV =Constant Value and LS =Length Scale.

Table 6
Optimal configuration for FCN model.

Distribution	# Neuron	Loss	Activation	# Epochs	Reg.
Uniform	16	MAE	Sigmoid	4000	L2
Gaussian	64	MSE	Sigmoid	4000	L2
Exponential	64	MSE	Sigmoid	4000	L2
Zipf	32	MAE	Sigmoid	500	L2

- **Learning rate:** This rate indicates the weights update strength in back-propagation in the gradient decent. Choosing this parameter is important for training since it may not converge with a too large value, while it will be slow to converge if the learning rate is too small. A typical value for learning rate is $1e-3$, which also used in our experiments.
- **Batch normalization:** The normalization layer is a key element to update the weights between neurons; that is, the weights are normalized at each update to the network. Our FCN model improves the regression performance with a batch normalization layer.

Fig. 6 illustrates our FCN model. The input layer takes the input data containing three features of data size, cache size, and distribution parameter value. As mentioned, the Batch Normalization (BN) layer is a key to optimize the overall performance and we observed a significant performance improvement with this intermediate layer. We evaluated a set of structures with different numbers of hidden layers but observed insignificant performance gaps among them; for simplicity, we chose a single hidden layer structure. The output is the predicted cache hit rate. We employed Adam as the optimizer in the implementation [22].

We conducted extensive experiments to evaluate the impact of the parameters, including the number of neurons, number of epochs, activation functions (Sigmoid and ReLU), loss functions (MAE and MSE), and regularization (L1 and L2). Table 6 shows the configuration performing the best for each distribution. Overall, we observed Sigmoid and L2 regularizer work slightly better than the other options. MAE works better for uniform and Zipf, whereas MSE is more a suitable choice for Gaussian and exponential. In our experiments, the number of neurons in the hidden layer do not make considerable impacts on the regression performance. We also observed that the epoch parameter has a slightly greater sensitivity than the number of neurons.

3.2.4. Comparison of prediction performance

We next report the performance comparison for the three regression models. We present a subset of the experimental results due to the space reason, but the other results also show almost the same trend with insignificant differences.

Fig. 7 shows cache hit rates over different cache sizes, under the assumption of the uniform access. Fig. 7(a) and 7(b) assume 3 GB and 6 GB for the data size to be accessed, respectively. The x-axis shows the cache size per server (hence, the aggregated cache size is three times of the cache size per node as we assume three cache servers in our experiments). While the regression models work quite well, we can see that FCN slightly outperforms the others.

Fig. 7(c) and 7(d) demonstrate cache hit rates for two tenants over the different cache size configurations with respect to the cache allocation ratio in the x-axis. That is, the ratio of 1:9 in x-axis indicates that 10% of the cache space is allocated to one tenant and the rest of the cache space (90% of the total space) is assigned to the other. The regression model used in this experiment is FCN. Fig. 7(c) compares the measured and predicted cache hit rates when using a 1 GB cache memory, while Fig. 7(d) shows the result with a 2 GB cache memory, under the assumption of

Table 7
Regression performance (MSE).

Distribution	SVR	GPR	FCN
Uniform	22.03	18.76	1.70
Gaussian	10.01	10.90	4.87
Exponential	4.90	5.35	0.66
Zipf	1.35	3.48	1.43

the uniform access. The plots show the regression performs very well for two independent tenants.

We also conducted a set of experiments for the non-uniform distributions and observed similar trends. We omit the presentation of the results due to the space reason. Table 7 summarizes the best performance of each regression model, in which we can see that SVR and GPR work poorly for the uniform distribution. A simple FCN model works consistently outperforming the other techniques. In terms of the training and prediction complexities, FCN showed greater overheads than SVR and GPR. The FCN training complexity is quite high (with no use of GPUs) showing two orders of magnitudes higher. Although the prediction overhead for FCN is slightly greater than the others, we observed that a single prediction could be made within 0.26 s on a commodity computer (equipped with Intel core i5).

3.3. Dynamic Cache resizing

As a result of the cache hit rate prediction, the cache management function determines whether the cache space needs to be resized or not. The procedure for resizing the tenant cache space is straightforward with the given distribution d , as follows:

- If $H(C_{T_k}, d) < H_{T_k} - \delta_1$, then the minimal cache space s such that $H(C_{T_k} + s, d) \geq H_{T_k} + \delta_1$ is additionally allocated to tenant k ;
- If $H(C_{T_k}, d) > H_{T_k} + \delta_2$, then the maximal cache space s such that $H(C_{T_k} - s, d) \geq H_{T_k} + \delta_2$ is returned from the tenant to the system;
- Otherwise, cache size for tenant k remains the same.

Here, δ parameters are configurable to consider safety margins. The worst scenario is no more cache space available in the system in case (i); if this is the case, we assume that the system needs to install more resources to expand the cache space in the system to meet the SLA goals for the entire tenants.

4. Evaluation

We designed a set of experiments to validate the operation of the dynamic cache management with the presented estimation and prediction functions in the previous section. In this section, we report our evaluation results conducted on a real cluster system, with (1) the synthetic traces based on different distributions, and (2) the YCSB benchmark tool. We first describe the experimental settings, and then discuss the experimental results with the metrics of cache hit rate and response time to measure the performance.

4.1. Experimental settings

We conducted our experiments in a computing cluster (elephant.tamuc.edu) that consists of 27 nodes mounted in a rack. Each node consists of 4 CPU cores, 8 GB memory, and 2 TB hard disk storage. The nodes are interconnected via a Gigabit Ethernet switch. We installed Apache Ignite 1.8.0 [4] and MariaDB [24] as the in-memory cache infrastructure and the backend DBMS, respectively. We configured three nodes for the in-memory cache

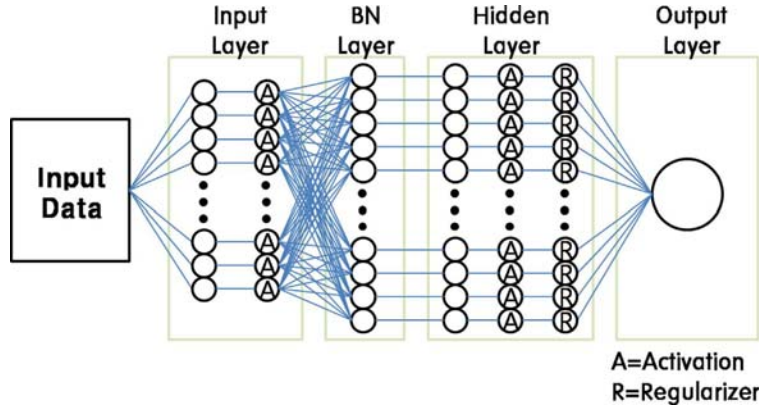


Fig. 6. The FCN structure for cache hit rate prediction: The input layer takes the input data containing three features of data size, cache size, and distribution parameter value. We chose a single hidden layer structure based on our evaluation results. The output is the predicted cache hit rate.

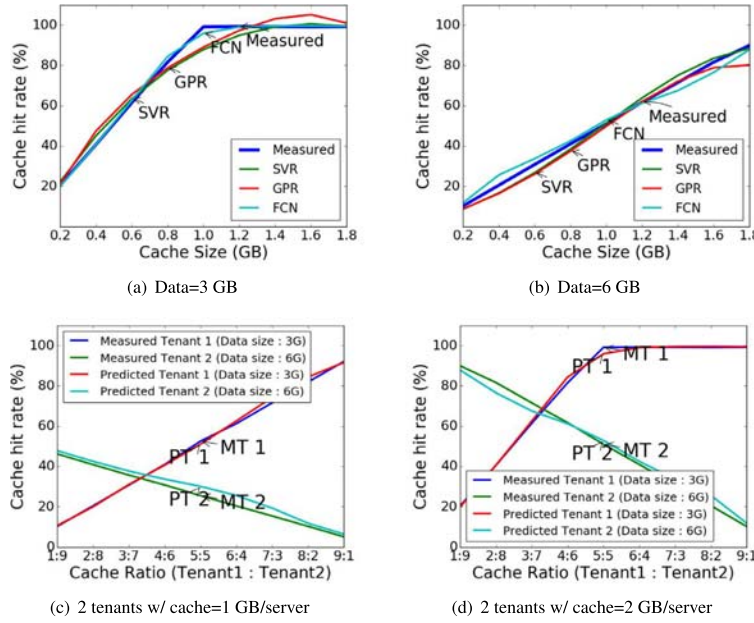


Fig. 7. Regression performance with Uniform distributions: (a) and (b) show the regression performance over different cache sizes for a tenant, and (c) and (d) show the performance for two tenants over different cache allocation ratios.

service, each of which is configured with 6 GB cache space, and hence, the total cache space is 18 GB in the system. Apache Ignite provides a set of built-in eviction methods including FCFS and LRU, and we simply chose LRU for cache replacement. One node is dedicated as a database server with MariaDB.

Fig. 8 illustrates the procedure for experiments, the main objective of which is to see if the proposed dynamic cache management is effective to meet the tenant's QoS requirement based on the data access pattern and the specified cache hit rate requirement. We assume that the data size is 3 GB for the tenant and the required cache hit rate is 80% at minimum. The initial cache size is 0.1 GB/node (i.e., 0.3 GB/system with three cache servers). Each experiment consists of N queries with a certain distribution (to generate keys), and $N=1$ million by default. For each query, the `get(key)` operation is invoked to look up the cache. In case of cache miss, the backend server is accessed to retrieve the entry associated with the given key from the database, and the `put(key, val)` operation is executed to add a new entry

to the cache. Once M queries are serviced, the system performs the estimation and prediction functions to see if resizing of cache space is needed. We set $M = \frac{N}{2}$ in our experiment to compare the performance before and after the event of cache resizing. Since the initial cache size is too small (0.3 GB) compared to the data size (3 GB), cache resizing will be triggered to meet the desired performance requirement. Note that the distribution estimation is performed with the KS-test and the prediction takes place using the FCN model, described in the previous section. Note that the parameter values used for FCN can be found from Table 6.

4.2. Experiments with synthetic traces

We first present the experimental results conducted with a set of synthetic traces with different distribution models. For the experiments, we installed Yardstick-ignite as a benchmark tool. Yardstick-Ignite provides 8 types of benchmark tests, and

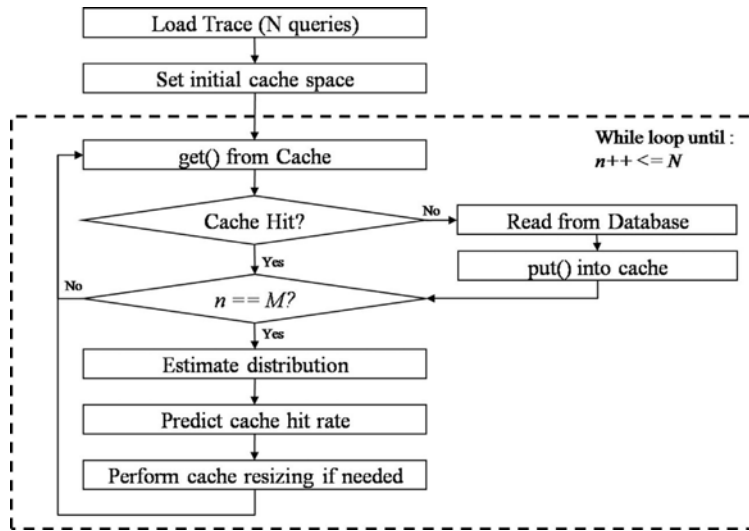


Fig. 8. Flow chart of the experiment procedure.

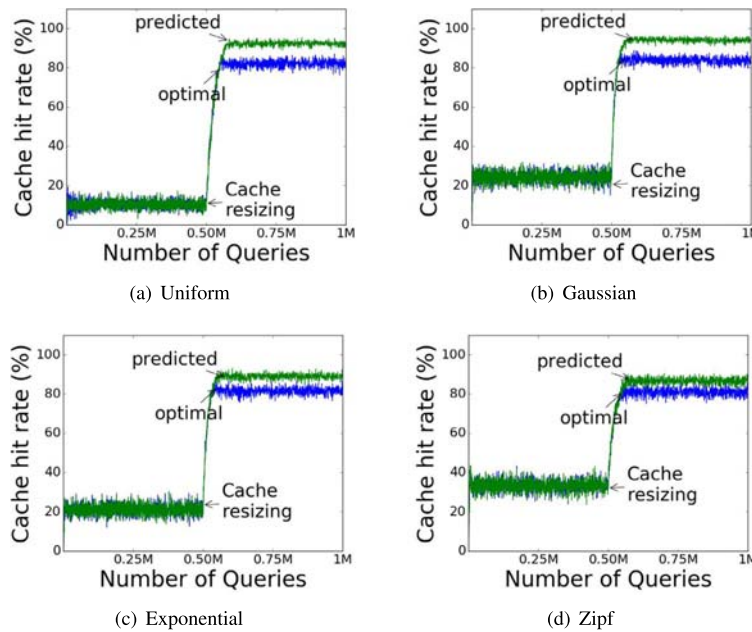
Fig. 9. Cache hit rates before and after resizing cache size with synthetic traces (minimum hit rate requirement=80%, $\delta=0.05$).

Table 8

Experimental setting for synthetic traces. Note that the testing traces were chosen from out of the training data sets.

Test case	Distribution	Data size	Parameter
T_1	Uniform	3 GB	–
T_2	Gaussian	3 GB	$\sigma = 0.7$
T_3	Exponential	3 GB	$\lambda = 0.7$
T_4	Zipf	3 GB	$\rho = 0.7$

we utilized PutGetTxBenchmark providing transactional distributed cache put and get operations. Table 8 shows the test cases prepared for the experiments with the synthetic data. As noted earlier, the data sets used for training and testing are

disjoint without any overlaps, as summarized in Tables 4 and 5. Thus, the data size (3 GB) and the distribution parameter values used for testing were chosen from outside the training data sets.

Fig. 9 shows the cache hit rates before and after the cache resizing. In the figure, “optimal” resizes the cache based on the measurement data without relying on the prediction, while “predicted” manages the cache based on our prediction procedure. For the prediction, we set the safety margin to 5% (i.e., $\delta=0.05$) based on the observation that the max difference between the measured and predicted hit rates is less than 0.05. With the safety margin, our prediction-based cache management will try to adjust the cache size to make the predicted hit rate to be $(80 + \delta)\%$ at minimum.

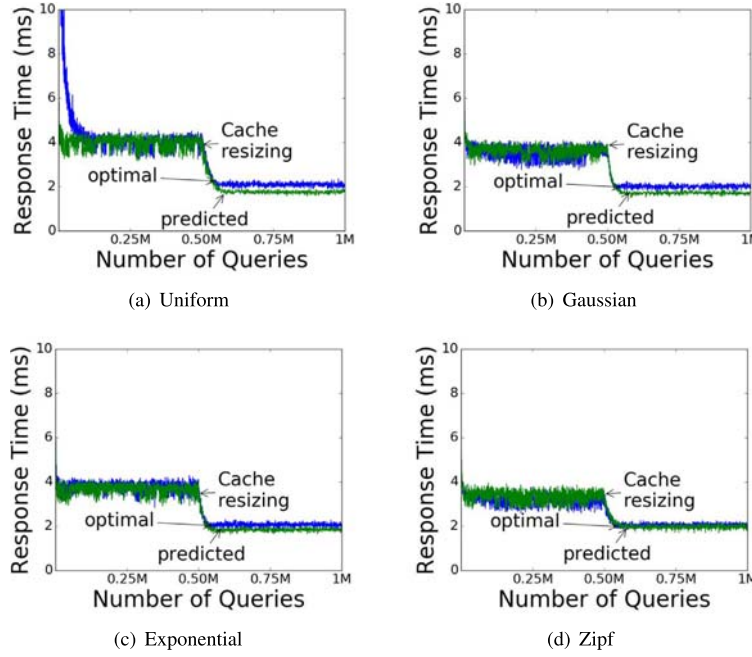


Fig. 10. Response times before and after resizing cache size with synthetic traces (minimum hit rate requirement=80%, $\delta=0.05$).

From the figure, the initial hit rates are very low. Based on the estimation of the access distribution and the prediction of the hit rate, the cache size is adjusted at $n=500,000$ queries, and the cache hit rates jump up to over 80% on average. The figure shows that the prediction works very well and allocates a slightly greater space for the cache compared to optimal (0.1 GB greater on average), due to the safety margin.

Fig. 10 shows the corresponding response time over the number of queries. In the initial warm up phase, the response time is very high since there is no entry in cache and every operation triggers the database access. As soon as the entire cache space is filled in, the average response time becomes stable. We can see that the response time significantly goes down after the cache is resized at $n=500,000$. The response time based on the prediction is slightly lower than optimal, since the prediction-based resizing allocates the more space to cache (and hence, with greater hit rates).

Tables 9 and 10 summarize the experimental results with the four synthetic data sets. We can see that our estimation process is able to identify the exact distribution and the associated parameter value. In the table, the measured hit rate and response time stand for the cache hit rate and response time after resizing the cache space based on the predicted cache size information, which meet the required hit rates.

We next assume consecutive changes of the data access patterns over time. In this experiment, the estimation process takes place at every 10,000 queries. If any pattern change is identified, the cache prediction process is activated and the cache space is reallocated if needed. The injected trace includes four different distributions in order: (1) exponential ($\lambda=0.9$), (2) Zipf ($\rho=1.1$), (3) uniform, and (4) Gaussian ($\sigma=1.3$). The initial cache size allocated is 0.1 GB, and the required hit rate is 80%. We simply set $\delta=0.0$ (i.e., no safety margin) in this experiment. The following lists the events and observations over the temporal pattern changes:

- (i) At $x=0$, the initial pattern follows exponential ($\lambda=0.9$);
- (ii) At $x=10,000$, the cache is resized to 0.7 GB (from 0.1 GB);

- (iii) At $x=250,000$, the pattern is changed to Zipf ($\rho=1.1$);
- (iv) At $x=260,000$, the cache size is reduced to 0.3 GB;
- (v) At $x=500,000$, the pattern is changed to uniform;
- (vi) At $x=510,000$, the cache size increases to 1.4 GB;
- (vii) At $x=750,000$, the pattern is changed to Gaussian ($\sigma = 1.3$);
- (viii) At $x=760,000$, the cache size is changed to 1.1 GB.

Fig. 11 demonstrates the dynamic cache management over time. Initially, the system identifies the access pattern and resizes the cache space to 0.7 GB to meet the QoS requirement. Although the cache space is adjusted at $x=10,000$, it takes a time to warm up the cache as shown in the figure. At every time that the access pattern is changed, we can see the degradation of the cache hit rate, but the performance is restored by dynamically resizing the cache space. At $x=260,000$ the cache size becomes shrunk (from 0.7 GB to 0.3 GB), but we can see that the cache hit rate meets the requirement. Fig. 11(b) shows the corresponding response time over the temporal changes.

4.3. Experiments with YCSB

We next report the experimental results conducted with YCSB, a benchmark tool widely used for evaluating the performance for RDBMS and NoSQL [14,26]. We slightly modified this benchmark tool to perform the experiments as specified in the experimental procedure in Fig. 8.

Fig. 13 demonstrates the cache hit rate and response time experimented with the YCSB tool. This experiment also compares the proposed method with the log-based concave function ($y = a + b \log x$) to estimate data access pattern discussed in Section 2.3. We set minimum hit rate requirement=80% and $\delta=0$ for this experiment. As the experiments with the synthetic traces, we initially allocate 0.1 GB/node for the cache space, and the estimation and prediction take place at $n=500,000$. We observed that the concave-based function calculates $a = 86.7$ and $b = 11.4$, which estimates 0.6 GB/node to meet the performance requirement as shown in Fig. 12. However, Fig. 13(a) reveals that it is an under-estimation, resulting in the lower hit rate than 80% after resizing.

Table 9

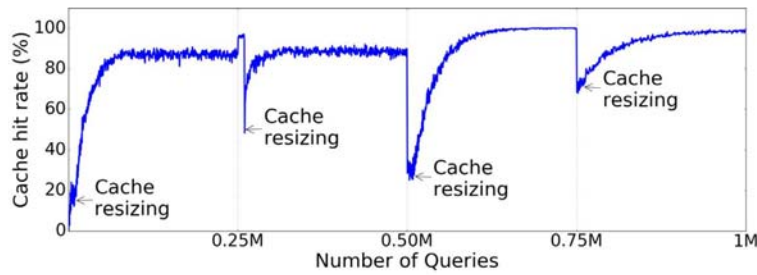
Experimental result with synthetic traces before resizing cache memory.

Test case	Initial cache size	Initial hit rate	Initial resp. time
T_1	0.1 GB	10.2%	4.58 ms
T_2	0.1 GB	24.1%	3.59 ms
T_3	0.1 GB	21.1%	3.67 ms
T_4	0.1 GB	33.2%	3.37 ms

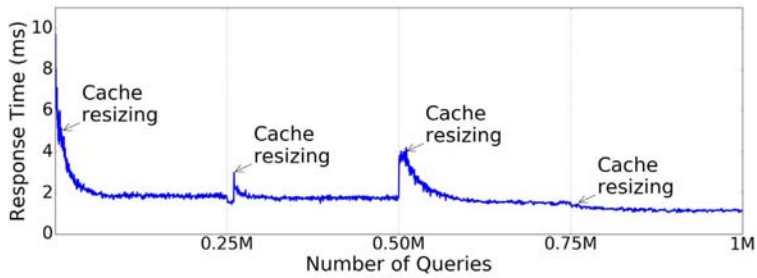
Table 10

Experimental result with synthetic traces after resizing cache memory.

Test case	Optimal cache size	Optimal hit rate	Estimated distribution	Predicted cache size	Measured hit rate	Measured resp. time
T_1	0.8 GB	81.8%	Uniform	0.9 GB	91.8%	1.77 ms
T_2	0.4 GB	83.7%	Gaussian(0.7)	0.5 GB	94.0%	1.78 ms
T_3	0.5 GB	80.6%	Exponential(0.7)	0.6 GB	88.7%	1.84 ms
T_4	0.6 GB	80.7%	Zipf(0.7)	0.7 GB	86.4%	1.96 ms



(a) Cache hit rate



(b) Response time

Fig. 11. Cache hit rate and response time over data access pattern changes: The injected trace includes four different distributions in order: (1) exponential ($\lambda=0.9$), (2) Zipf ($\rho=1.1$), (3) uniform, and (4) Gaussian ($\sigma=1.3$). The initial cache size is 0.1 GB, the required hit rate is 80%, and $\delta=0$.

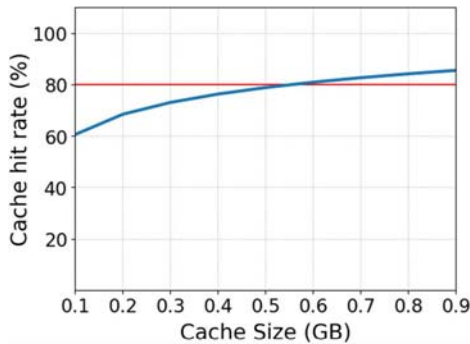


Fig. 12. Estimation of cache space based on the log-based concave function ($y = 86.7 + 11.4\log x$): The estimation shows that it needs 0.6 GB to meet 80% hit rate (when assuming 0.1 GB as the block unit for the allocation).

In contrast, our proposed method estimates the distribution as Zipf with $\rho=1.0$, and the cache is resized to 0.8 GB/node as a result

of the prediction through FCN. In Fig. 13(c), we can see that the resized cache space based on our method meets the performance requirement. The measured cache hit rate after resizing is 75.8% on average with 1.89 msec of mean response time when using the concave-based technique, while our proposed method yields 80.3% hit rate on average with 1.66 msec of mean response time meeting the specified requirement.

5. Conclusions

The in-memory cache has been widely employed to improve data access performance in a cloud. Despite its importance, the past studies were largely limited with the lack of well-defined models for the estimation of data access patterns and the prediction of cache performance for the access pattern in question. In this paper, we proposed a learning-based approach to dynamic caching in a cloud to meet the per-tenant QoS requirement. We first presented an estimation method that approximates the data access pattern to one of four distribution models of uniform, Gaussian, exponential, and Zipf, based on the KS test. We observed that our estimation method works well with a high degree

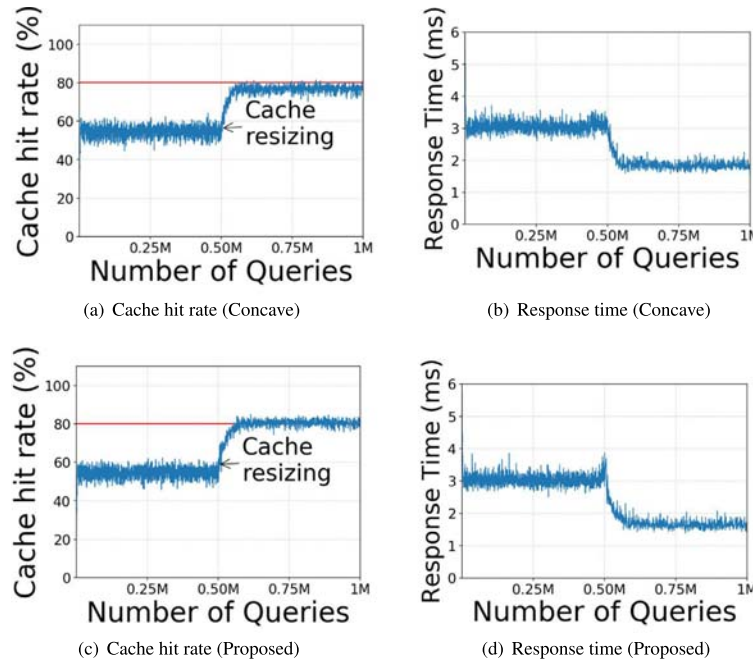


Fig. 13. Cache hit rate and response time before/after resizing cache size using YCSB benchmark (minimum hit rate requirement=80% and $\delta=0$): (a) estimation by the log-based concave function ($y = a + b \log x$), and (b) estimation by the proposed method.

of accuracy even with a small number of query samples (≥ 200 samples), which should be beneficial for responding to the temporal pattern changes in a timely manner. We next presented the evaluation results of a set of regression methods including SVR, GPR, and FCN, to predict the cache hit rate based on the estimated access pattern. From the experiments, we observed that the FCN model outperforms the others across the distributions. Finally, we evaluated our dynamic cache management method with an extensive set of synthetic traces and the YCSB benchmark. The evaluation results show that the proposed method consistently optimizes the cache space, while preserving the tenant's QoS requirement.

The cloud cache management consists of a set of functions and this work focused on the problem of cache space optimization. Another important function in the cache management is cache eviction that has a significant impact on the data access performance. In this work, we assumed the traditional LRU policy for eviction, but the estimated access pattern would be the helpful information to improve cache hit rates. A planned future task is to investigate an adaptive method for cache eviction over access pattern changes. In addition, it is assumed a fixed length of cache slots to simplify the problem, but multiple slab classes may need to be considered for different types of applications requiring various slot sizes.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] H. Ahlehagh, S. Dey, Video caching in radio access network: Impact on delay and capacity, in: 2012 IEEE Wireless Communications and Networking Conference, WCNC, IEEE, 2012, pp. 2276–2281.
- [2] G. Almási, C. Çaşcaval, D.A. Padua, Calculating stack distances efficiently, *SIGPLAN Not.* 38 (Suppl. 2) (2002) 37–43.
- [3] Amazon ElastiCache, <https://aws.amazon.com/elasticache/>.
- [4] Apache ignite in-memory platform, <https://ignite.apache.org/>.
- [5] S. Baek, D. Kwon, J. Kim, S. Suh, H. Kim, I. Kim, Unsupervised labeling for supervised anomaly detection in enterprise and cloud networks, in: Proceedings of the 4th IEEE International Conference on Cyber Security and Cloud Computing, IEEE CSCloud 2017, 2017, New York, NY, USA.
- [6] L. Breslau, P. Cao, L. Fan, G. Phillips, S. Shenker, Web caching and zipf-like distributions: Evidence and implications, in: INFOCOM, 1999.
- [7] M.C. Calzarossa, L. Massari, D. Tessera, Workload characterization: A survey revisited, *ACM Comput. Surv.* 48 (3) (2016) 48:1–48:43, <http://dx.doi.org/10.1145/2856127>.
- [8] M. Chen, Y. Hao, L. Hu, K. Huang, V.K. Lau, Green and mobility-aware caching in 5g networks, *IEEE Trans. Wirel. Commun.* 16 (12) (2017) 8347.
- [9] L. Cherkasova, M. Gupta, Analysis of enterprise media server workloads: access patterns, locality, content evolution, and rates of change, *IEEE/ACM Trans. Netw.* 12 (5) (2004) 781–794.
- [10] G. Chockler, G. Laden, Y. Vigfusson, Data caching as a cloud service, in: Proceedings of the 4th International Workshop on Large Scale Distributed Systems and Middleware, LADIS '10, ACM, New York, NY, USA, 2010, pp. 18–21.
- [11] G.V. Chockler, G. Laden, Y. Vigfusson, Design and implementation of caching services in the cloud, *IBM J. Res. Dev.* 55 (6) (2011) 9.
- [12] J. Choi, K. Hu, A. Sim, Relational Dynamic Bayesian Networks with Locally Exchangeable Measures, Tech. Rep. LBNL-6341E, Lawrence Berkeley National Laboratory, 2013.
- [13] A. Cidon, A. Eisenman, M. Alizadeh, S. Katti, Dynacache: Dynamic cloud caching, in: Proceedings of the 7th USENIX Conference on Hot Topics in Cloud Computing, HotCloud'15, 2015, p. 19.
- [14] B.F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC '10, ACM, New York, NY, USA, 2010, pp. 143–154.
- [15] B. Fitzpatrick, Distributed caching with memcached, *Linux J.* 2004 (124) (2004) 5.
- [16] N. Golrezaei, A.F. Molisch, A.G. Dimakis, G. Caire, Femtocaching and device-to-device collaboration: A new architecture for wireless video distribution, *IEEE Commun. Mag.* 51 (4) (2013) 142–149.
- [17] GPR, http://scikit-learn.org/stable/modules/gaussian_process.html.
- [18] D. Gu, H. Hu, Spatial Gaussian process regression with mobile sensor networks, *IEEE Trans. Neural Netw. Learn. Syst.* 23 (8) (2012) 1279–1290.

- [19] K.P. Gummadi, R.J. Dunn, S. Saroiu, S.D. Gribble, H.M. Levy, J. Zahorjan, Measurement, modeling, and analysis of a peer-to-peer file-sharing workload, *SIGOPS Oper. Syst. Rev.* 37 (5) (2003) 314–329, <http://dx.doi.org/10.1145/1165389.945475>.
- [20] L. Guo, E. Tan, S. Chen, Z. Xiao, X. Zhang, The stretched exponential distribution of internet media access patterns, in: *Proceedings of the Twenty-Seventh ACM Symposium on Principles of Distributed Computing, PODC '08*, ACM, New York, NY, USA, 2008, pp. 283–294.
- [21] U.U. Hafeez, M. Wajahat, A. Gandhi, Elmem: Towards an elastic memcached system, in: *ICDCS, 2018 Proceedings, IEEE, 2018*, pp. 278–289.
- [22] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, CoRR [arXiv:abs/1412.6980](https://arxiv.org/abs/1412.6980).
- [23] P. Lu, K. Shen, Virtual machine memory access tracing with hypervisor exclusive cache, in: *Usenix Annual Technical Conference, 2007*, pp. 29–43.
- [24] MariaDB, <https://mariadb.org/>.
- [25] MemCachier, <https://www.memcachier.com/>.
- [26] Q. Pu, H. Li, M. Zaharia, A. Ghodsi, I. Stoica, FairRide: Near-optimal, fair cache sharing, in: *Proceedings of the 13th Usenix Conference on Networked Systems Design and Implementation, NSDI'16, 2016*, pp. 393–406.
- [27] Y. Rahulamathavan, R.C. Phan, S. Veluru, K. Cumanan, M. Rajarajan, Privacy-preserving multi-class support vector machine for outsourcing the data classification in cloud, *IEEE Trans. Dependable Sec. Comput.* 11 (5) (2014) 467–479.
- [28] L. Ramaswamy, L. Liu, A. Iyengar, Cache clouds: Cooperative caching of dynamic documents in edge networks, in: *25th IEEE International Conference on Distributed Computing Systems, ICDCS'05, IEEE, 2005*, pp. 229–238.
- [29] Redis, <https://redis.io/>.
- [30] D. Shue, M.J. Freedman, A. Shaikh, Performance isolation and fairness for multi-tenant cloud storage, in: *Presented As Part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation, {OSDI} 12, 2012*, pp. 349–362.
- [31] Y.S. Sun, Y.-F. Chen, M.C. Chen, A workload analysis of live event broadcast service in cloud, *Procedia Comput. Sci.* 19 (2013) 1028–1033.
- [32] B. Wang, S. Sen, M. Adler, D. Towsley, Optimal proxy cache allocation for efficient streaming media distribution, in: *Proceedings. Twenty-First Annual Joint Conference of the IEEE Computer and Communications Societies, vol. 3, IEEE, 2002*, pp. 1726–1735.
- [33] Y. Yu, W. Wang, J. Zhang, Q. Weng, K.B. Letaief, Opus: Fair and efficient cache sharing for in-memory data analytics, in: *ICDCS, 2018 Proceedings, IEEE, 2018*, pp. 154–164.
- [34] W. Zhou, Y. Wang, Y. Xiang, J. Zhang, Y. Xiang, Y. Guan, Network traffic classification using correlation information, *IEEE Trans. Parallel Distrib. Syst.* 24 (2013) 104–117.



Mr. Jinhwan Choi received his Master degree in Computer Science from Texas A&M University-Commerce in 2018. He is currently a data scientist of Cleveland Clinic Foundation. After obtaining his Master degree, he was a data scientist/bigdata platform engineer of SK holdings and C&C, South Korea. He worked on several deep learning projects and led the platform team for deep learning modeler. Before obtaining his Master degree, he worked for Samsung SDS as a software engineer from 2007 to 2016. He worked on several projects, including the video analysis platform, Samsung DTV platform, and digital signage solution.



Dr. Yu Gu received his Ph.D. degree in Computer Science from University of Minnesota, Twin Cities. He is currently with Visa and leading the Visa AI Platform. The areas of research interests span from mini sensor networks to hyper scale cloud and distributed systems. Prior to that, he was a research scientist at IBM Research Austin and Watson Health from 2014 to 2017, and assistant professor at Singapore University of Technology and Design from 2010 to 2014.



Dr. Jinoh Kim received his Ph.D. degree in Computer Science from University of Minnesota, Twin Cities. He is currently an Associate Professor in the Department of Computer Science at Texas A&M University-Commerce. His research interests include cybersecurity, data-intensive computing, big-data computing, and distributed systems. He was a postdoctoral researcher at the Lawrence Berkeley National Laboratory for 2010–2011 and an Assistant Professor of Computer Science at Lock Haven University of Pennsylvania for 2011–2012. Prior to that, he was a researcher and a senior researcher at ETRI, Korea since 1991, and participated in various projects in the areas of network security and broadband Networks.